T1-01: The Modern AI System Map

DLS 2026

Draft — 11 October 2026

Learning objective. Given a proposed AI feature, students will be able to draw the data, model, evaluation, deployment, and governance handoffs; name the evidence that must cross each handoff; and identify where a failure should be diagnosed.

1 A model is not the system

A trained neural network is a parameterized function. An AI product is a socio-technical system that decides what input to collect, how to represent it, which model and prompt to run, whether to retrieve or call tools, what answer to display, and how to monitor consequences. This distinction matters because an impressive offline model score does not establish that a deployed workflow is useful, safe, affordable, or reversible. The transformer is one component of a system, not the system itself [1, 3].

A useful map has five connected planes: *intent and policy*; *data and training*; *model and post-training*; *serving and interaction*; and *measurement and governance*. Each plane has an owner, artifacts, assumptions, and failure modes. A handoff is sound only when the next plane can inspect the evidence needed to decide whether to proceed.

2 The lifecycle and its evidence

1. Intent. State the user, task, decision authority, harm model, and success criterion. "Make a helpful assistant" is not testable; "help students explain a derivation while labeling uncertainty and refusing to invent citations" is closer. This stage creates a requirements document and a threat model.

2. Data. Acquire, document, clean, partition, and govern examples. Data cards should record provenance, permissions, sampling boundaries, transformations, and known gaps. Split design is a causal guardrail: if near-duplicates appear in training and test sets, a claimed generalization result may merely be leakage.

3. Training and adaptation. Pre-training learns broad statistical structure; supervised instruction tuning and preference or outcome-based optimization shape behavior for a task. These stages change tradeoffs rather than granting a blanket guarantee of truthfulness or safety [2]. Record model version, code revision, random seeds where practical, training mixture, and evaluation exclusions.

4. Evaluation. Offline evaluation asks whether a model meets a specified criterion under a defined distribution. It should include representative tasks, adversarial or stress cases, calibrated uncertainty, and error analysis—not a single aggregate score. A benchmark measures a construct only to the extent its task, scoring, and contamination controls support that interpretation.

5. Serving. The runtime path may include authentication, a system prompt, retrieval, tool calls, moderation, caching, rate limits, observability, and a user interface. Serving changes the realized system: retrieval can improve grounding but introduce stale or malicious sources; caching reduces cost but can violate freshness expectations; an agent can take useful actions but needs permissions and recovery logic.

6. Monitoring and governance. Deployment is an experiment with real consequences. Measure latency, cost, availability, user outcomes, error categories, and incidents. Preserve an audit trail sufficient

to reproduce a decision. NIST's risk framework emphasizes governing, mapping, measuring, and managing risks throughout this lifecycle rather than treating safety as a final filter [4].

3 Worked example: a course-help assistant

Suppose a course assistant answers questions about a lab. The intent is to coach rather than supply hidden solutions. The data plane contains the released assignment, rubric, and permitted course notes; it excludes private grading records. The model plane selects a versioned model and prompt. The serving plane retrieves only the released materials, labels citations, blocks file-system writes, and logs a minimal privacy-preserving trace. Evaluation includes correct-help cases, ambiguous questions, prompt-injection attempts in uploaded text, and questions whose right response is "ask the instructor." Monitoring checks whether students can complete legitimate setup steps and whether answers create the same misconception repeatedly.

Notice the handoffs. A retrieval corpus owner must certify what is current; the evaluator must test the actual tool-enabled path rather than a bare chat completion; the instructor must decide what assistance is permitted. If the assistant invents a command, the repair could be retrieval quality, prompt policy, tool schema, model selection, or user-interface framing. Calling it simply a "hallucination problem" hides the system diagnosis.

4 Design exercise: trace a handoff

Take a proposed campus navigation assistant. A student asks for an accessible route to a building during construction. The *intent* artifact states that the answer is advisory, not emergency direction, and identifies the facilities team as authority for closures. The *data* artifact lists maps, accessibility annotations, construction notices, update times, and licenses. The *model* artifact records the model version and prompt policy. The *runtime* artifact records which sources were retrieved, whether a mapping tool was called, and what fallback occurs when sources conflict. The *evaluation* artifact contains ordinary route queries, stale-notice cases, inaccessible-route traps, and cases requiring a human referral. The *governance* artifact identifies who can disable the feature and how an incident is reported.

This exercise exposes why names of components are not enough. Suppose the assistant recommends a closed elevator. A useful incident report asks whether the closure feed was missing, stale, incorrectly retrieved, summarized incorrectly, overridden by a cache, or presented ambiguously in the interface. Different diagnoses imply different repairs and different owners. A system map makes this inquiry possible before a public failure forces it.

5 A practical evidence checklist

Before a system change, require a compact change record: (1) purpose and non-goals; (2) input sources, permissions, and retention; (3) versions of model, prompts, tools, and retrieval corpus; (4) tests executed and their failure categories; (5) expected performance, latency, and cost boundaries; (6) monitoring signals and alert ownership; and (7) rollback or containment action. The record should be proportionate to the consequence of the action. A spelling assistant and an agent allowed to modify records do not deserve the same authorization process.

The checklist also helps distinguish uncertainty from omission. It is acceptable to say that the effect of a new retrieval source is unknown and run a limited evaluation. It is not acceptable to deploy without being able to say which source was used, who approved access, or how to turn the feature off. In instructional work, this same discipline helps students separate a plausible diagram from evidence about a real system.

6 Limits and uncertainties

The map is not a compliance checklist and cannot prove a system harmless. Evidence is contextual: a benchmark score can shift when users, tools, or distributions change. Some risks are normative, such as who may authorize consequential actions, and require human governance rather than an optimization target. The map is valuable because it makes assumptions visible and assigns an accountable next action.

7 Takeaway

When evaluating an AI proposal, ask: What is the decision? What data and permissions support it? Which versioned runtime path produces it? What evidence demonstrates performance and failure boundaries? Who can stop or change the system? A defensible AI system answers all five.

References

[1] Vaswani et al. (2017), *Attention Is All You Need*.

[2] Ouyang et al. (2022), *Training language models to follow instructions with human feedback*.

[3] Huyen (2022), *Machine Learning Systems Design*.

[4] NIST (2023), *AI Risk Management Framework 1.0*.